Primer Proyecto: Servidor de simulaciones electromagnéticas FDTD concurrente

CI-0117 - Programación Paralela y Concurrente

Universidad de Costa Rica

Andrés Oreamuno Varela

Octubre 2026

1. Introducción

El problema propuesto consiste en simular una onda electromagnética en una malla 2D con el método FDTD, donde cada paso de tiempo actualiza tres campos: Hx, Hy y Ez. Con este solucionador se atenderán a muchos clientes, manteniendo trabajos en cola y permitiendo consultar el progreso y los resultados de los trabajos.

El proyecto implementa un servidor escrito en C que recibe solicitudes de simulaciones FDTD de múltiples clientes vía TCP y las procesa mediante un solucionador paralelizado con Pthreads. Se utilizan diversos mecanismos de sincronización vistos en el curso (mutex, semáforos, barreras) y se pretende analizar el rendimiento utilizando la ley de Amdahl.

2. Desarrollo

El trabajo consta de tres ejecutables: fdtd_server, fdtd_client y fdtd_bench. El servidor se encuentra escuchando peticiones de parte de los clientes vía línea de comandos y las responde.

Para montar el servidor se utilizó como base el proyecto mini_server facilitado por el profesor, así como las tareas realizadas relacionadas con semáforos y mutex.

2.1. Cola de trabajos y despachador

Se implementó una cola circular de capacidad 8, protegida por 1 mutex y 2 semáforos: el mutex asegura los accesos de lectura y escritura mientras que los semáforos indican cuántos campos libres y ocupados existen. En esta cola se implementa el patrón productor consumidor, el consumidor es un único hilo despachador.

Para cumplir con el enunciado del proyecto se utilizó la función sem_trywait sobre el semáforo de campos libres, así si la cola está llena el trabajo no se queda esperando y retorna BUSY, evitando que el cliente se bloquee y garantizando así no se pierda el trabajo.

2.2. Solucionador FDTD

La malla se reparte por filas entre los T hilos (descomposición de dominio). En cada paso de tiempo, cada hilo actualiza primero los campos Hx y Hy de sus filas y luego Ez. Dentro de una misma fase ninguna celda depende de otras celdas de esa fase, por lo que los hilos pueden trabajar al mismo tiempo. Sin embargo, para actualizar sus filas de borde, un hilo necesita leer celdas de las filas vecinas, que pertenecen a otro hilo; por eso entre una fase y otra se usa una barrera.

Se usan dos barreras por paso: después de actualizar H y después de actualizar Ez. La barrera es reutilizable y se implementó con un mutex, un contador y dos semáforos (dos torniquetes). No se utilizó pthread_barrier_t.

Con la opción de compilación `make SIN_BARRERA=1` se define `SIN_BARRERA` y la función de barrera retorna sin esperar.

2.3. Progreso, PEEK y RESULT

El solucionador publica su avance en una estructura compartida con el paso actual y una copia (instantánea) de Ez. Después de la segunda barrera de cada paso, el hilo 0 actualiza el paso y, cada k pasos, copia la malla Ez. Esta estructura se protege con un candado de lectores-escritor hecho con semáforos, basado en la tarea corta 5: el hilo 0 es el escritor y STATUS y PEEK son lectores. RESULT devuelve la suma del valor absoluto del campo Ez en todas las celdas de la malla final.

2.4. Apagado

El proyecto implementa un apagado ordenado, primero deja de aceptar conexiones, espera a que terminen los manejadores activos y limpia la cola (los trabajos en espera se descartan). Finalmente libera los recursos.

3. Justificación a los requisitos de sincronización

1. **¿Por qué su barrera es reutilizable y no permite que un hilo rápido se adelante un paso completo?**

   La barrera tiene dos fases. En la primera, cada hilo incrementa el contador, el último en llegar abre el primer torniquete para todos. En la segunda, cada hilo decrementa el contador, el último en salir abre el segundo torniquete. Un hilo rápido que termina la primera fase no puede volver a entrar a la barrera del siguiente paso hasta que todos hayan pasado por la segunda fase, porque queda detenido en el segundo torniquete. Así se puede usar la misma barrera para ambos pasos.

2. **¿Por qué el hilo 0 puede copiar Ez mientras los demás hilos ya actualizan Hx y Hy del paso siguiente?**

   Porque la copia se hace después de la segunda barrera, cuando todos los hilos terminaron de escribir Ez. En la fase siguiente los demás hilos actualizan Hx y Hy, que solo leen Ez. Si la copia se hiciera durante la fase de Ez, la instantánea podría mezclar celdas del paso anterior con celdas del paso actual.

3. **¿Qué política usa el candado de lectores-escritor y cómo evita que muchos PEEK dejen al solucionador sin publicar?**

   Se usa la solución sin inanición del escritor: además del contador de lectores y el semáforo de sala vacía, hay un torniquete. Cuando el escritor (hilo 0) quiere publicar, toma el torniquete; los nuevos lectores (PEEK, STATUS) quedan detenidos en él y solo esperan a que salgan los lectores que ya

estaban adentro. Así se garantiza que el escritor tendrá prioridad sobre los nuevos lectores.

4. **¿Cómo se garantiza que SUBMIT nunca bloquee al cliente y que no se pierdan ni dupliquen trabajos?**

   SUBMIT usa sem_trywait, que no espera: si no hay espacio, retorna de inmediato y avisa que el servidor está BUSY al cliente. El id se asigna y el trabajo se inserta dentro del mutex de la cola, por lo que dos manejadores no pueden recibir el mismo id ni escribir en la misma casilla.

5. **¿Qué ocurre con los hilos bloqueados en semáforos durante SHUTDOWN?**

   Solo el despachador puede estar bloqueado en un semáforo, los hilos solucionadores estarán en la barrera a lo sumo pero podrán terminar su ciclo. Para despertar al hilo despachador se encola una píldora, al sacarla, sale de su ciclo y el hilo principal lo espera con pthread_join. Antes de encolar la píldora se espera a que no haya manejadores activos y se vacía la cola, por lo que siempre hay espacio para la píldora.

6. **Con la opción -DSIN_BARRERA, ¿qué diferencias se esperan y por qué no siempre aparecen?**

   Sin barrera, un hilo puede leer celdas vecinas que otro hilo todavía no ha actualizado (o que ya actualizó para el paso siguiente), por lo que el resultado puede variar entre corridas. No siempre aparece la diferencia porque depende del orden en que el sistema operativo ejecute los hilos.

4. Pruebas

Se realizaron pruebas manuales con el cliente y el servidor para corroborar el funcionamiento de SUBMIT, STATUS, PEEK, RESULT y SHUTDOWN.

Para medir el rendimiento se usó fdtd_bench, que ejecuta el solucionador sin el servidor, y el script `scripts/bench.sh`, que corre cada caso 5 veces. Se midieron dos escenarios con 1, 2, 4 y 8 hilos:

- E1: malla de 2048×2048, 1000 pasos.
- E3: malla de 256×256, 1000 pasos.

Las pruebas se corrieron en una máquina con procesador Intel Atom Z3736F (4 núcleos físicos, 1 hilo por núcleo, 1.33 GHz hasta 2.17 GHz), dos cachés L2 de 2 MiB compartidas cada una por un par de núcleos y 1.8 GiB de RAM. Los datos están en `evidencia/ambiente_Bo.txt` y `evidencia/bench_Bo.csv`.

En todas las corridas, la suma de la malla final fue la misma con 1, 2, 4 y 8 hilos: $3{,}1124951568 \times 10^2$ en E1 y $1{,}7225053013 \times 10^2$ en E3.

5. Análisis de resultados

Se reporta la mediana de las 5 veces que se repitió cada caso.

Escenario	Hilos	T (s)	S	Amdahl	E	e(n)
E1	1	81.90	1.00	1.00	1.00	–
E1	2	60.17	1.36	1.99	0.68	0.47
E1	4	68.23	1.20	3.97	0.30	0.78
E1	8	67.81	1.21	7.86	0.15	0.80
E3	1	1.31	1.00	1.00	1.00	–
E3	2	0.74	1.77	1.99	0.88	0.13
E3	4	2.30	0.57	3.95	0.14	2.02
E3	8	3.78	0.35	7.79	0.04	3.16

Cuadro 1: Tiempos, speedup, Amdahl, eficiencia y Karp–Flatt.

En E1 el mejor resultado fue con 2 hilos ($S = 1{,}36$); con 4 y 8 hilos el speedup se queda cerca de 1.2. En E3, con 2 hilos el speedup (1.77) se acerca al de Amdahl (1.99), pero con 4 y 8 hilos el programa es más lento que con 1 hilo.

La diferencia no se explica por la parte serial: la métrica de Karp–Flatt crece al aumentar los hilos, lo que indica que la pérdida viene del costo de paralelizar y no de una fracción serial fija.

6. Conclusiones

La barrera garantiza el mismo resultado con cualquier número de hilos.

Más hilos no siempre es más rápido: aquí el mejor caso fue con 2 hilos.

El límite no fue la parte serial (Amdahl), sino la memoria y la sincronización (Karp–Flatt).

Quedaron pendientes el búfer de instantáneas con hilo escritor a disco, el escenario E2 y las pruebas automáticas. También pruebas en distinto hardware.

7. Repositorio

`https://github.com/andresoreamuno/fdtd-server`

Referencias

[1] A. B. Downey, *The Little Book of Semaphores*, 2.ª ed. Green Tea Press, 2016. [En línea]. Disponible: `https://greenteapress.com/wp/semaphores/`

[2] P. S. Pacheco, *An Introduction to Parallel Programming*. Burlington, MA, EE. UU.: Morgan Kaufmann, 2011.